# Growth of Functions

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

# GROWTH OF FUNCTION

- **Ex)**
  - ✓ Insertion Sort: $\Theta(n^2)$
  - ✓ Merge Sort: $\Theta(n \lg n)$

  - ✓ Questions
    - "Which one grows faster as $n$ increases?"
    - "What is $\Theta(\cdot)$?"
    - "How do we get the $\Theta(\cdot)$ of each algorithm?"

# Growth of Function

- **Algorithm's efficiency**
  - ✓ can be evaluated by **growth of function**.
  - ✓ allows us to compare the relative performance of algorithms.
  - ✓ Instead of determining the exact running time of an algorithm, we will see the **asymptotic efficiency** focusing on what's important.
    - – We usually abstract away low-order terms and constant factors.

# Asymptotic Efficiency

## ❖ Asymptotic Efficiency

- ✓ How efficient an algorithm is when the input size becomes very large.
- ✓ Cf) asymptote (점근선)

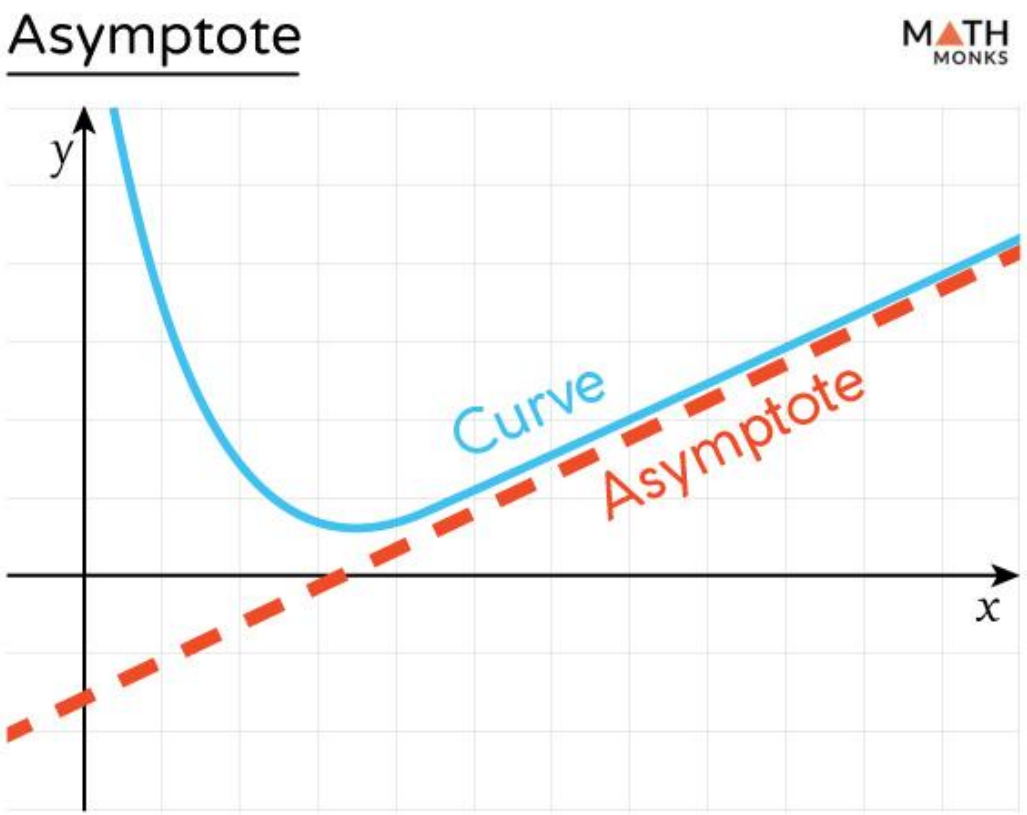
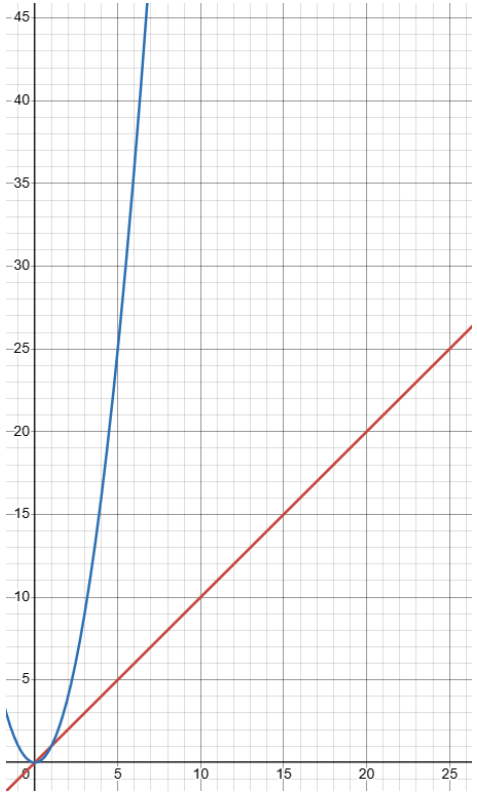



Image credit: https://mathmonks.com/asymptote

## ❖ Asymptotic Efficiency

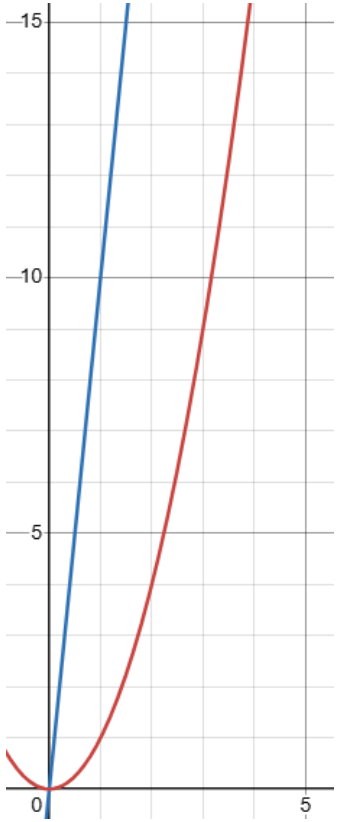
- ✓ How efficient an algorithm is when the input size becomes very large.
- ✓ Ex) $n$ (red line) vs. $n^2$ (blue line)
  - – Which one grows faster as $n$ increases?

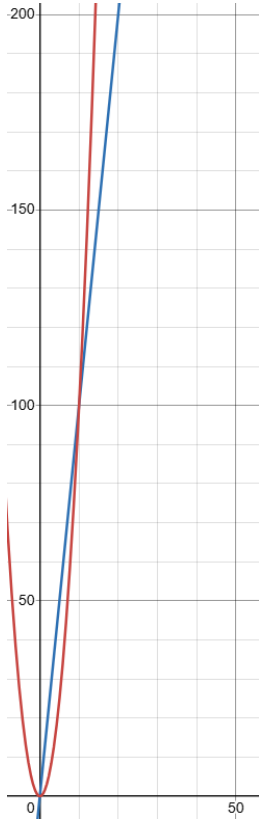



https://www.desmos.com/calculator

## ❖ Asymptotic Efficiency

- ✓ How efficient an algorithm is when the input size becomes very large.
- ✓ Ex) $10 \cdot n$ (red line) vs. $n^2$ (blue line)
  - – Which one grows faster as $n$ increases?




https://www.desmos.com/calculator

- **Asymptotic Efficiency**
  - ✓ How efficient an algorithm is when the input size becomes very large.
  - ✓ Ex) what about $1000000000000000000000000000000000000000000000 \cdot n$ vs. $n^2$?

**As $n$ grows,**
**$n^2$ always eventually beats**
**any linear function($c \cdot n$)**
**no matter what a constant value $c$ is.**

- **Asymptotic Efficiency**
  - ✓ How efficient an algorithm is when the input size becomes very large.

**So, we say,**
**$n^2$ is asymptotically less efficient than $n$ or vice versa.**

- **Asymptotic Efficiency**
  - ✓ How efficient an algorithm is when the input size becomes very large.

# One more thing:
# All linear function($c \cdot n$) have the same asymptotic efficiency!

- **Let's use mathematical expressions to describe the asymptotic efficiency!**

- **Set of functions**
  - ✓ $\{ f : f \text{ satisfies some condition}\}$
  - ✓ Ex)

    $\{ f : f \text{ grows asymptotically as fast as } n\}$

    $= \{n, 1.1n, 2n, 100n, \dots\}$ (actually, there is more beautiful way to describe this set, which will be covered soon)

    $= \Theta(n)$ (Theta of $n$)

    Some more….

    $2n \in \Theta(n)$ ($2n$ is in the set of functions, $\{ f : f \text{ grows asymptotically as fast as } n\}$)

    You can also think of $\Theta(n)$, $\Theta(n^2)$, $\Theta(\log(n))$, $\Theta(2^n)$, and so on.

    Thus, $\Theta(g(n))$ is one of the ways to express the set of functions, $\{ f : f \text{ grows asymptotically as fast as } g(n)\}$.

- **How can we describe the relation between asymptotical efficiency of $n$ and $n^2$?**
  - ✓ Intuitively and simply, you can imagine that $n^2$ stays above $n$.
  - ✓ So, you can say $n^2$ is an upper bound for $n$.

- ✓ $n^3$ is also an upper bound for $n$.
- ✓ $n^{100}$ is also an upper bound for $n$.
- ✓ $n^{100}$ is also an upper bound for $n^2$.
- ✓ $n^{100}$ is also an upper bound for $n^3$.

- ✓ $n \in O(n^3)$
- ✓ $n \in O(n^{100})$
- ✓ $n^2 \in O(n^{100})$
- ✓ $n^3 \in O(n^{100})$

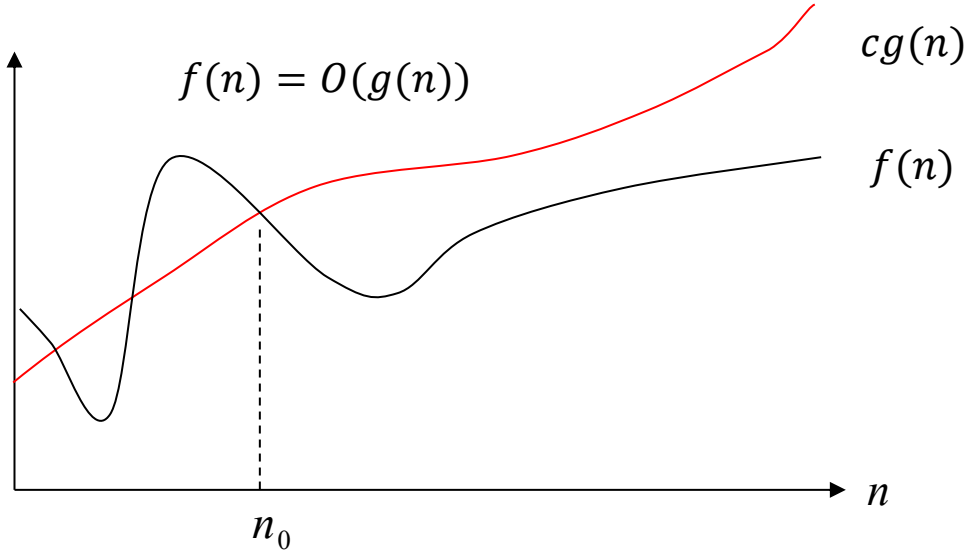
- ✓ $O(n^{100}) = \{f : n^{100} \text{ is an upper bound for } f\}$

$$= \{n, n^2, n^3, \dots, n^{4.6}, \dots, n^{99}, \dots\}$$

$$= \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \le f(n) \le c \cdot n^{100} \text{ for all } n \ge n_0\}$$

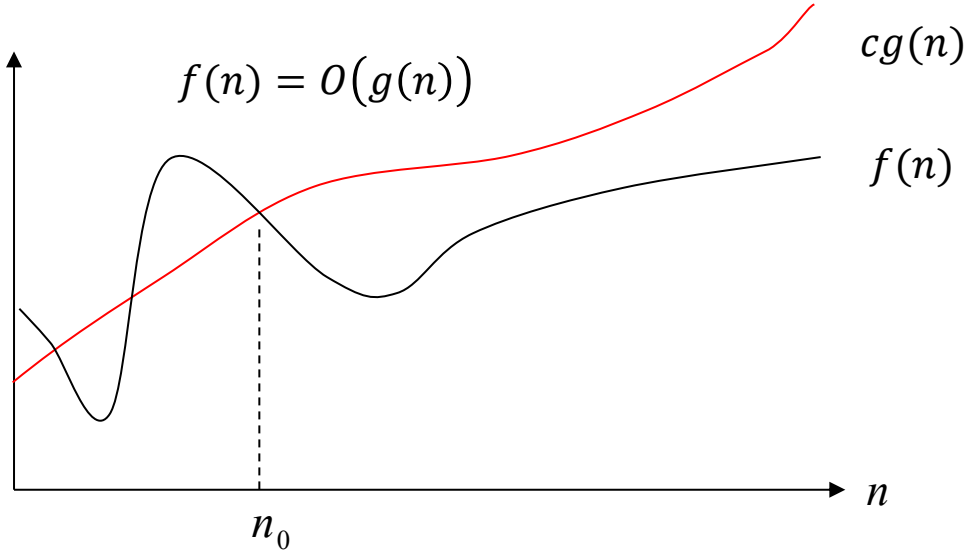
## ❖ Upper Bound ($O$-notation)

- ✓ Ex) $O(n^{100}) = \{f(n)$: there exist positive constants $c$ and $n_0$ such that $0 \le f(n) \le c \cdot n^{100}$ for all $n \ge n_0\}$
- ✓ More generally,
  $O(g(n)) = \{f(n)$: there exist positive constants $c$ and $n_0$ such that $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0\}$



## ❖ Upper Bound (Big $O$-notation)

- ✓ $O(g(n)) = \{f(n)$: there exist positive constants $c$ and $n_0$ such that $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0\}$
- ✓ We also say, simply $f(n) = O(g(n))$ if $f(n) \in O(g(n))$.
- ✓ A function is also an upper bound for itself.
  - $f(n) \in O(f(n))$ or $f(n) = O(f(n))$

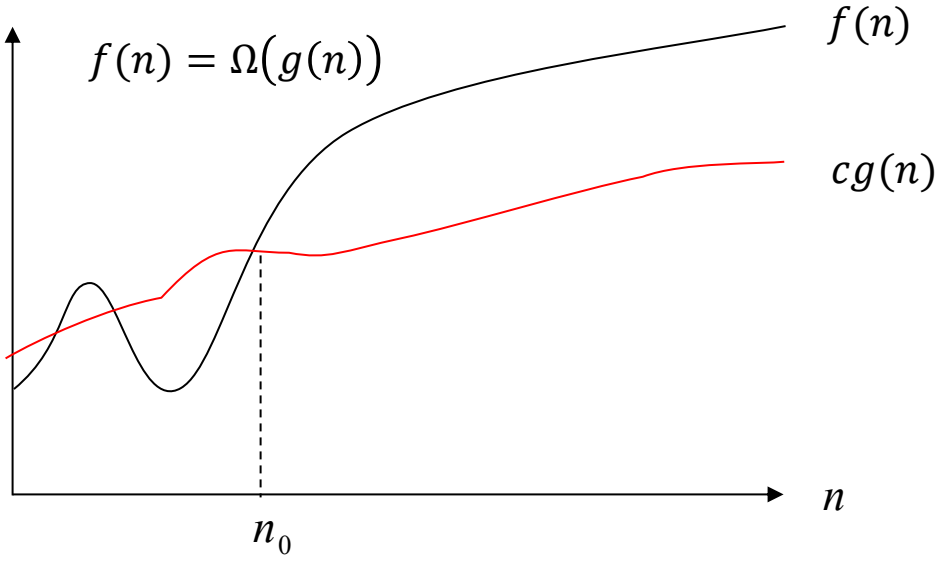



$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \text{constant} < \infty$$

$$0 \le f(n) \le c \cdot g(n)$$

## ❖ Lower Bound (Big Omega Ω-notation)

- ✓ $\Omega(g(n)) = \{f(n)$: there exist positive constants $c$ and $n_0$ such that $0 \le c \cdot g(n) \le f(n)$ for all $n \ge n_0\}$
- ✓ We also say, simply $f(n) = \Omega(g(n))$ if $f(n) \in \Omega(g(n))$.
- ✓ A function is also an upper bound for itself.
  - $f(n) \in \Omega(f(n))$ or $f(n) = \Omega(f(n))$




$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \text{constant} > 0$$

$$0 \le c \cdot g(n) \le f(n)$$

## ❖ Tight (Exact) Bound (Big Theta $\Theta$-notation)

- ✓ $\Theta(g(n))$
  $= \{f(n)$: there exist positive constants $c_1, c_2$, and $n_0$ such that $0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0\}$
- ✓ We also say, simply $f(n) = \Theta\big(g(n)\big)$ if $f(n) \in \Theta\big(g(n)\big)$.
- ✓ Ex) $\frac{n^2}{2} - 2n = \Theta\big(n^2\big)$
  - We usually abstract away low-order terms and constant factors.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \text{constant} > 0 \text{ and } < \infty$$

For two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$
if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

- **Polynomial functions are easy to compare…**
  **but what about other non-polynomial functions???**
  - ✓ such as $\sqrt{n}, \ln(n), e^n, n^e$…

- **L'Hôpital's Rule**
  - ✓ Using the L'Hôpital's Rule, we can figure out which functions goes to infinity faster!

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{f'(n)}{g'(n)}$$

  - ✓ Watch this video "Growth Rates of Functions and L'Hospital's Rule"
    - https://youtu.be/6IjVGfbhe9w

❖ **Notation Summary** (simplified for easier memorization)

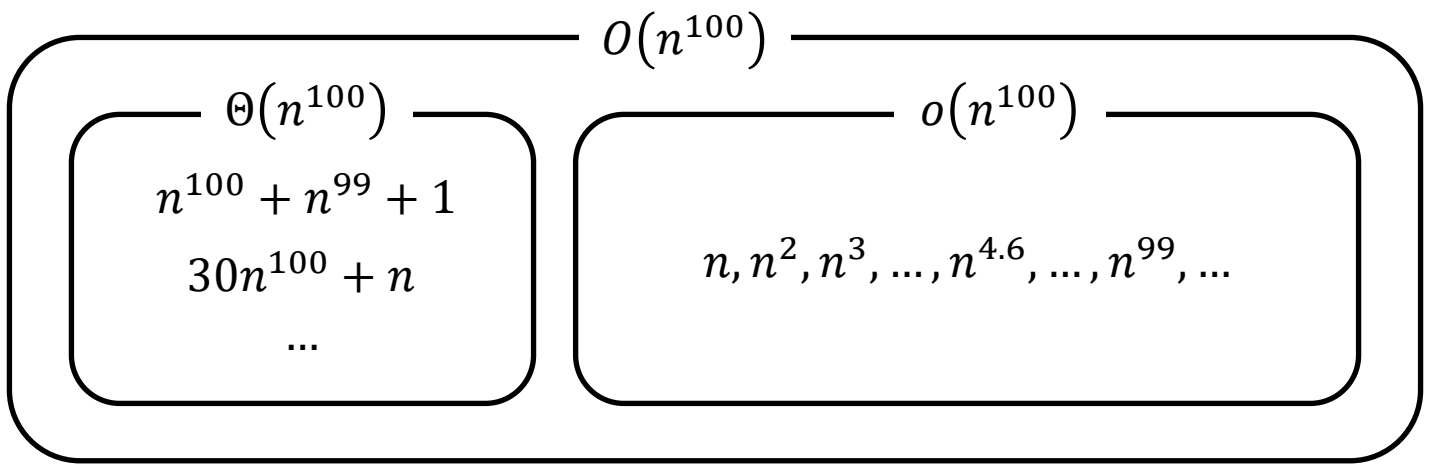
| | | |
|---|---|---|
| **Theta** | $f(n) = \theta(g(n))$ | $f(n) \approx c\, g(n)$ |
| **Big Oh** | $f(n) = O(g(n))$ | $f(n) \leq c\, g(n)$ |
| **Big Omega** | $f(n) = \Omega(g(n))$ | $f(n) \geq c\, g(n)$ |
| | | |
| | | |

$c$ = positive constant value

## ❖ Small $o$-notation

- ✓ $o(g(n)) = \{f(n)$: there exist positive constants $c$ and $n_0$ such that $0 \le f(n) < c \cdot g(n)$ for all $n \ge n_0\}$
- ✓ $f(n)$ is ***asymptotically smaller*** than $g(n)$ and $f(n) \in o(g(n))$ if $f(n) = o(g(n))$.
- ✓ $f(n) \notin o\big(f(n)\big)$
- ✓ $f(n) \neq o\big(f(n)\big)$
- ✓ $o\big(f(n)\big) = O\big(f(n)\big) - \Theta\big(f(n)\big)$ (set subtraction)

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \text{ (previously, it was constant} < \infty \text{ for Big O notation)}$$



## ❖ Small $\omega$-notation

- ✓ $\omega(g(n)) = \{f(n)$: there exist positive constants $c$ and $n_0$ such that $0 \leq c \cdot g(n) < f(n)$ for all $n \geq n_0\}$
- ✓ $f(n)$ is ***asymptotically larger*** than $g(n)$ and $f(n) \in \omega(g(n))$ if $f(n) = \omega(g(n))$.
- ✓ $f(n) \notin \omega\big(f(n)\big)$
- ✓ $f(n) \neq \omega\big(f(n)\big)$
- ✓ $\omega\big(f(n)\big) = \Omega\big(f(n)\big) - \Theta\big(f(n)\big)$ (set subtraction)

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty \text{ (previously, it was constant} > 0 \text{ for Big } \Omega \text{ notation)}$$

❖ **Notation Summary** (simplified for easier memorization)

| | | |
|---|---|---|
| **Theta** | $f(n) = \theta(g(n))$ | $f(n) \approx c\, g(n)$ |
| **Big Oh** | $f(n) = O(g(n))$ | $f(n) \leq c\, g(n)$ |
| **Big Omega** | $f(n) = \Omega(g(n))$ | $f(n) \geq c\, g(n)$ |
| **Little Oh** | $f(n) = o(g(n))$ | $f(n) < c\, g(n)$ |
| **Little Omega** | $f(n) = \omega(g(n))$ | $f(n) > c\, g(n)$ |

$c$ = positive constant value

## ❖ Simplest form

- ✓ Abstracting away low-order terms and constant factors, we get the simplest form.
- ✓ Ex)

$\Theta((8n^3 + n)^{1/2} + \log n)$

$= \Theta(2n^{3/2} + 6n \ln n)$

$= \Theta(n^{3/2})$ ← simplest form

$= \Theta(n^{3/2} + 23n^{1/3})$

$= \Theta(\frac{n^3+n+1}{4n^{3/2}+\ln n})$

- **More interesting properties of $\Theta$**
  - ✓ Reflexive
    - $f(n) \in \Theta(f(n))$
  - ✓ Symmetric
    - $f(n) \in \Theta\big(g(n)\big) \leftrightarrow g(n) \in \Theta(f(n))$
  - ✓ Transitive
    - $f(n) \in \Theta(g(n))$ and $g(n) \in \Theta\big(h(n)\big) \rightarrow f(n) \in \Theta(h(n))$

- **What about $\Omega, \omega, \mathrm{O}, o$?**

## ❖ Usage of the notations for analyzing the time complexity of an algorithm

- ✓ $O(\cdot)$:
  Helps find the upper bound on the worst-case running time of an algorithm.
- ✓ $\Omega(\cdot)$:
  Helps find lower bound on the best case of running time of an algorithm.
- ✓ $\theta(\cdot)$:
  Do both.

## ❖ Insertion Sort

- ✓ Running time $T(n)$ is a quadratic function in the worst case. So, $T(n) = \Theta(n^2)$.
- ✓ Running time $T(n)$ is a linear function in the best case. So, $T(n) = \Theta(n)$.
- ✓ So, $T(n) = O(n^2)$ and $T(n) = \Omega(n)$ for all case.
- ✓ The worst-case time complexity of Insertion Sort doesn't match the lower bound of the problem.

## ❖ Optimality of an algorithm

- ✓ When the worst-case time complexity of an algorithm matches the lower bound of the problem, the algorithm is said to be optimal.
- ✓ The worst-case time complexity of Insertion Sort doesn't match the lower bound of the problem.
  - So, Insertion Sort is not an optimal algorithm.
- ✓ "Optimal" does not mean "the best known"; it means "the best possible".

## ❖ Merge Sort

- ✓ It is known as Running Time $T(n) = \Theta(n \log n)$, which means also $T(n) = \mathrm{O}(n \log n)$.
- ✓ It is also proved that any sorting algorithm cannot have a lower bound lower than $\Theta(n \log n)$.
- ✓ Thus, Merge Sort's Running Time $T(n) = O(n \log n)$ and $T(n) = \Omega(n \log n)$.
- ✓ Merge Sort is an optimal algorithm.

- **One last word**
  - ✓ Even though Insertion Sort is not an optimal solution, it can be a practically useful algorithm for small $n$!

Thank you!